

데이터 요약과 시계열 처리

Outline

거래 8건을 정리해 매장 담당자에게 전달할 범주·월별 매출 요약표를 만든다.

중심 질문

- 여러 거래 행을 범주와 시간별 시각화 입력으로 어떻게 요약하는가?
- 요약 과정에서 행 수·합계·결측의 의미가 보존됐음을 어떻게 확인하는가?

학습 순서

- 문자열 정리 → 범주별 묶기와 요약 → 두 범주의 교차 요약 순서로 익힌다.
- 시간값 변환 → 시간 간격별 요약 → 이동 비교 → 전체 파이프라인 검산으로 마무리한다.

거래 기록 DataFrame 생성

실행 가능한 공통 입력

- 매장 담당자의 입력은 거래 8건이다. event_id는 거래 ID, occurred_at은 시점, category는 범주다.
- item_tag는 상품|태그, channel은 경로, sales는 매출(천 원), units는 수량(개)이다. 이 셀부터 순서대로 실행한다.

셀 구조와 확인 결과

- pd는 pandas, np는 NumPy를 가리킨다. 바깥 []는 행 목록, 각 ()는 한 거래의 일곱 값이다.
- columns는 일곱 열 이름을 붙인다. None은 상품 정보 결측이다. 총매출 136을 범주·월·경로별로 요약한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
import numpy as np
import pandas as pd

sales = pd.DataFrame([
    (1, "2026-01-03 09:00", " Drink ", "Latte|HOT", "web", 12, 1),
    (2, "2026-01-10 10:00", "drink", "Tea|iced", "store", 18, 2),
    (3, "2026-01-31 21:00", "Food", "Sandwich|cold", "web", 25, 2),
    (4, "2026-02-01 08:00", "food ", "Soup|hot", "store", 15, 1),
    (5, "2026-02-14 12:00", "DRINK", None, "web", 20, 2),
    (6, "2026-02-28 18:00", "Snack", "Cookie|sweet", "store", 10, 1),
    (7, "2026-03-01 07:00", "snack ", "Chips|salty", "web", 14, 1),
    (8, "2026-03-08 16:00", "FOOD", "Salad|cold", "store", 22, 2),
], columns=["event_id", "occurred_at", "category", "item_tag",
            "channel", "sales", "units"])
```

03 · PANDAS

열 전체에 같은 규칙을 적용하는 벡터화 문자열 연산

공백과 대소문자 정리

- `.astype("string")` 은 열을 pandas 문자열 자료형으로 명시하고 결측을 문자열로 오인하지 않게 한다.
- `.str.strip()` 은 양끝 공백을 제거하고
`.str.lower()` 는 영문 대문자를 소문자로 바꾼다.
- 각 행에 반복문을 쓰지 않아도 같은 규칙이 열의 모든 값과 결측에 적용된다.

정리 후의 범주 판정

- 원문 그대로 묶으면 " Drink ", "drink", "DRINK" 가 갈라진다. 정리 후 세 거래의 매출은 $12+18+20=50$ 으로 모인다.
- 벡터화는 같은 연산을 열 전체에 적용하는 인터페이스다.
`head()` 는 처음 5행만 보여 주며 행의 순서·거래 ID는 바꾸지 않는다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
sales["category_clean"] = (
    sales["category"].astype("string").str.strip().str.lower()
)
print(sales[["category", "category_clean"]].head())
```

TEXT · 실행 결과

	category	category_clean
0	Drink	drink
1	drink	drink
2	Food	food
3	food	food
4	DRINK	drink

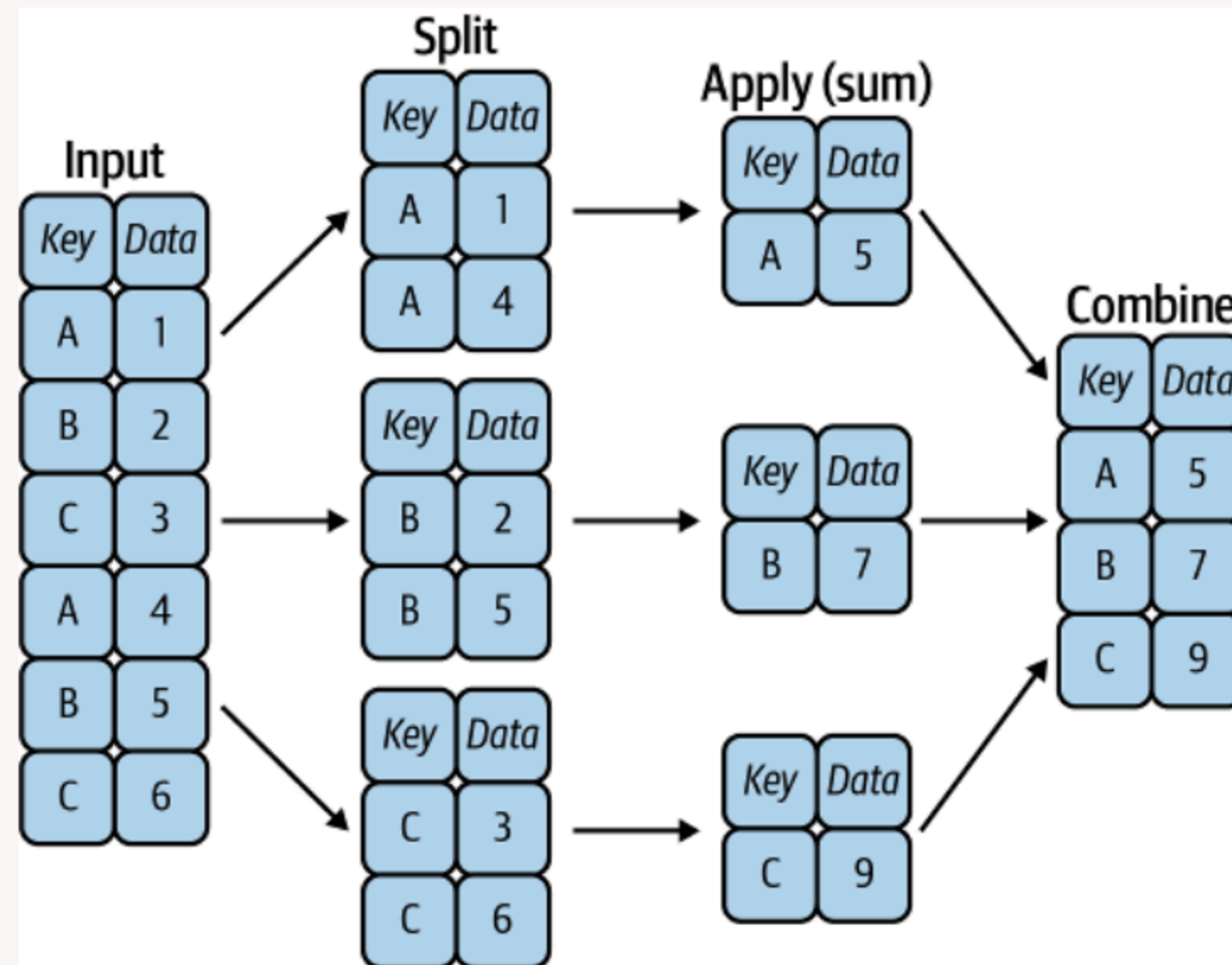
groupby의 분할·적용·결합 구조

세 단계의 의미

- 분할은 키가 같은 행을 묶고, 적용은 각 묶음에 합계 같은 함수를 계산한다.
- 결합은 그룹별 결과를 하나의 새 표로 모으며 중간 그룹 표는 직접 만들 필요가 없다.
- 왼쪽 입력 → 분할 → 합계 적용 → 오른쪽 결합 순으로 읽는다. 그림은 A·B·C 키로 6행을 나눈 뒤 각 그룹의 Data를 합치는 순서다.

출력 계약

- 집계 뒤에는 그룹 하나가 결과 한 행이 되고 A: 1+4=5, B: 2+5=7, C: 3+6=9이다.



교재 Figure 20-1 · 분할-적용-결합

05 · PANDAS

그룹별 합계·건수·평균

같은 범주를 묶어 한 행으로 요약

- `groupby` 는 같은 키의 거래를 묶는다.
`as_index=False` 는 키를 일반 열로 남긴다.
- `agg(결과열=(입력열, 함수))` 에서 `size` 는 건수,
`sum` 은 합계, `mean` 은 평균이다.
- 8행 → 3행, 범주 합계는 50·62·24이며 총합은 136이다.

결측과 평균의 분모

- 매출 [12, 결측, 20]이면 `size=3`, 유효값 수 `count=2`,
평균은 16이다.
- 그룹 키의 결측은 기본적으로 제외된다. 이 입력은 범주 결
측이 없는지 먼저 검사한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
assert sales["category_clean"].notna().all()
summary = sales.groupby("category_clean", as_index=False).agg(
    event_count=("event_id", "size"),
    total_sales=("sales", "sum"),
    mean_sales=("sales", "mean"),
)
totals = summary[["category_clean", "total_sales"]].rename(
    columns={"total_sales": "sales"})
print(summary.to_string(index=False))
```

실행 결과

category_clean	event_count	total_sales	mean_sales
drink	3	50	16.666667
food	3	62	20.666667
snack	2	24	12.000000

06 · PANDAS

집계·변환·필터의 출력 행 수

원래 거래마다 그룹 기준값이 필요할 때

- 앞의 집계는 그룹당 한 행이다. `transform("sum")` 은 그룹 합계를 원래 8행에 반복해 돌려준다.
- drink 거래 비율은 `12/50=0.24`, `18/50=0.36` 이다.

조건에 맞는 그룹 전체를 남길 때

- `filter` 는 그룹 단위 참·거짓으로 포함 여부를 정한다. `lambda g` 는 그룹을 받아 `len(g)` 로 행 수를 센다.
- 3건 이상인 drink·food의 원래 6행만 남는다. snack 2행은 함께 제외된다.
- 집계 3행·변환 8행·필터 6행: 필요한 결과 모양에 따라 선택한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
group_total = sales.groupby("category_clean")
["sales"].transform("sum")
share = sales["sales"] / group_total
kept = sales.groupby("category_clean").filter(lambda g: len(g)
>= 3)
print("그룹 합계:", group_total.tolist())
print("앞 두 비율:", share.head(2).tolist())
print("남은 ID:", kept["event_id"].tolist())
```

실행 결과

그룹 합계: [50, 50, 62, 62, 50, 24, 24, 62]
 앞 두 비율: [0.24, 0.36]
 남은 ID: [1, 2, 3, 4, 5, 8]

07 · PANDAS

두 범주를 행과 열로 배치하는 피벗 테이블

범주와 판매 경로의 교차 요약

- `values` 는 더할 값, `index` 는 행 범주, `columns` 는 열 범주를 지정한다.
- 기본 집계는 평균이다. `aggfunc="sum"` 은 같은 범주·경로 조합에 속하는 모든 거래의 매출을 더한다.
- 결과의 한 셀은 특정 범주와 특정 판매 경로가 만나는 매출 합계다.

합계와 기본 평균의 차이

- `drink×web` 입력은 `[12,20]` 이다. 기본 평균은 `16`, 매출 합계는 `32` 이므로 보고 목적에 맞춰 집계를 지정한다. `margins=True`를 추가하면 All 행·열에 주변 합계를 표시한다. 범주 50·62·24, 경로 65·71의 총합은 136이다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
matrix = sales.pivot_table(
    values="sales",
    index="category_clean",
    columns="channel",
    aggfunc="sum",
)
print(matrix)
```

TEXT · 실행 결과

channel	store	web
category_clean		
drink	18	32
food	37	25
snack	10	14

문자열 분해와 결측을 포함한 조건

상품과 태그를 별도 열로 분해

- `split("|", expand=True)` 는 구분자 양쪽을 별도 열로 만든다. 기본 `False` 는 한 셀에 리스트를 반환한다.
- 이 입력은 구분자가 한 개다. 두 열에 item·tag 이름을 붙이며 결측은 두 열 모두 <NA>로 남는다.

부분 일치와 정확한 태그의 차이

- `contains` 는 부분 문자열 검사다. `case=False` 는 대소문자 무시, `na=False` 는 결측을 미일치로 정한다.
- 앞 5행에서 HOT와 hot은 참, 결측은 거짓이다. 단, hotdog|cold도 부분 일치는 참이다.
- 태그만 찾으려면 분해한 tag를 소문자로 바꿔 `=="hot"` 으로 비교한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
tags = sales["item_tag"].astype("string")
parsed = tags.str.split("|", expand=True)
parsed.columns = ["item", "tag"]
hot = tags.str.contains("hot", case=False, na=False)
print(parsed.head().to_string(index=False))
print(hot.head().tolist())
```

실행 결과

```
item tag
Latte HOT
Tea iced
Sandwich cold
Soup hot
<NA> <NA>
[True, False, False, True, False]
```

시점·기간·간격과 시간 격자

시간값의 세 가지 의미

- `Timestamp` 는 한 시점, `Period` 는 한 달 같은 기간, `Timedelta` 는 이들 같은 시간 간격이다.
- 2월 1일 09:30에 이들을 더하면 2월 3일 09:30이다. 월 기간은 2월 전체를 가리킨다.

예상 시점의 목록 생성

- `date_range` 에 시작일, `periods=3` (개수), `freq="D"` (하루 간격)를 지정한다.
- 3월 1·2·3일 격자를 만든다. 정기 관측과 달리 불규칙 거래에서는 빈 날짜가 곧 기록 누락은 아니다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
stamp = pd.Timestamp("2026-02-01 09:30")
period = pd.Period("2026-02", freq="M")
delta = pd.Timedelta(days=2)
days = pd.date_range("2026-03-01", periods=3, freq="D")
print(stamp + delta)
print(period)
print(days.strftime("%Y-%m-%d").tolist())
```

실행 결과

```
2026-02-03 09:30:00
2026-02
['2026-03-01', '2026-03-02', '2026-03-03']
```

날짜 변환과 실패·결측 검사

문자열을 계산 가능한 시점으로 변환

- `to_datetime` 은 날짜 문자열을 시점으로 바꾼다. 거래 시점을 변환하고 결측이 없는지 확인한다.
- 기본 `errors="raise"` 는 잘못된 날짜에서 중단한다. `coerce` 는 변환 실패를 시간 결측 `NaT` 로 만들어 조사하게 한다.

중단과 조사는 목적에 따른 대안

- 2월 30일은 존재하지 않아 `coerce` 결과가 `NaT`다. `isna().sum()` 은 결측 개수를 센다.
- `raise`도 원래 결측은 `NaT`로 반환할 수 있다. 날짜가 필수라면 변환 후 결측 검사도 필요하다.
- 두 번 변환이 필수는 아니다. `NaT`를 발견했다고 검증이 끝난 것도 아니며 원자료를 조사해야 한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
sales["occurred_at"] = pd.to_datetime(sales["occurred_at"])
assert sales["occurred_at"].notna().all()
probe = pd.Series(["2026-02-28", "2026-02-30"])
checked = pd.to_datetime(probe, errors="coerce")
print(checked)
print("invalid:", checked.isna().sum())
```

실행 결과

```
0    2026-02-28
1           NaT
dtype: datetime64[ns]
invalid: 1
```

시간 범위 선택 기준인 DatetimeIndex

시점 열을 정렬된 인덱스로

- `.set_index("occurred_at")` 은 거래 시점을 행 선택 기준으로 만들고 `.sort_index()` 는 시간 순서를 보장한다.
- 부분 문자열 `"2026-02"` 는 2월 시작부터 끝까지의 시점을 포함한다.
- 결과 3행의 매출 `15 + 20 + 10 = 45` 가 2월 월별 집계 입력이다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
ts = sales.set_index("occurred_at").sort_index()
print(type(ts.index).__name__)
print(ts.loc["2026-02", ["event_id", "sales"]])
```

TEXT · 실행 결과

DatetimeIndex

	event_id	sales
occurred_at		
2026-02-01 08:00:00	4	15
2026-02-14 12:00:00	5	20
2026-02-28 18:00:00	6	10

12 · PANDAS

같은 시간 구간의 관측을 요약하는 resample

월 시작 간격으로 매출 합계 만들기

- `.resample("MS")` 는 `DatetimeIndex` 의 시점을 월 시작 기준 구간으로 나눈다. MS는 월 시작이다. 1월 구간은 1월 1일 이상·2월 1일 미만이며 결과 라벨은 1월 1일이다.
- `.sum()` 은 같은 월 구간의 거래를 더하고 결과 인덱스를 각 달의 시작일로 표시한다.
- 1월 55, 2월 45, 3월 36을 더한 136이 원자료 합계와 일치한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
monthly = ts["sales"].resample("MS").sum()
print(monthly)
```

TEXT · 실행 결과

```
occurred_at
2026-01-01    55
2026-02-01    45
2026-03-01    36
Freq: MS, Name: sales, dtype: int64
```

resample의 구간 집계와 asfreq의 격자 선택

같은 3일 입력의 다른 결과

- `resample("2D").sum()` 은 1·2일을 한 구간으로 묶어 $12 + 20 = 32$ 를 만든다.
- `asfreq("2D")` 는 1일과 3일의 기존 관측만 선택하므로 값은 12와 18이다.
- 구간 통계가 필요하면 `resample`, 달라진 시간 격자의 관측값이 필요하면 `asfreq` 를 쓴다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
daily = pd.Series(
    [12, 20, 18],
    index=pd.to_datetime(["2026-03-01", "2026-03-02", "2026-03-03"]),
)
print(daily.resample("2D").sum())
print(daily.asfreq("2D"))
```

실행 결과

```
2026-03-01    32
2026-03-03    18
Freq: 2D, dtype: int64
2026-03-01    12
2026-03-03    18
Freq: 2D, dtype: int64
```

빈 시간 칸의 채우기 방향과 가정

앞값 채우기와 뒤값 채우기

- `asfreq("D")` 는 3월 2일 칸을 만들고 관측이 없으므로 `NaN` 을 둔다.
- `ffill` 은 직전 12가 유지됐다고, `bfill` 은 다음 18을 앞당겨 쓸 수 있다고 가정한다.
- 실제 매출 미발생과 기록 누락은 다르므로 두 방법 중 하나를 자동 선택하면 안 된다. `fillna(0)`은 앞·뒤값 채우기와 달리 빈 칸을 0으로 가정한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
sparse = pd.Series(
    [12, 18],
    index=pd.to_datetime(["2026-03-01", "2026-03-03"]),
).asfreq("D")
print(pd.DataFrame({
    "raw": sparse, "ffill": sparse.ffill(), "bfill":
    sparse.bfill()
})))
```

실행 결과

	raw	ffill	bfill
2026-03-01	12.0	12.0	12.0
2026-03-02	NaN	12.0	18.0
2026-03-03	18.0	18.0	18.0

직전 값 비교와 두 관측 이동 평균

shift: 같은 행에 직전 값 배치

- `shift(1)` 은 인덱스를 유지하고 값을 한 행 아래로 옮긴다. `shift(-1)` 은 다음 값을 가져온다.
- 변화량은 2월 `45-55=-10`, 3월 `36-45=-9` 다. 첫 달은 비교 대상이 없어 NaN이다.

rolling: 창 안의 값을 요약

- `rolling(2).mean()` 은 현재·직전 행 평균이다. 기본 `min_periods=2`, `center=False` 로 두 값이 필요하다.
- 2월 `(55+45)/2=50`, 3월 `(45+36)/2=40.5` 다. 첫 NaN을 0으로 채우지 않는다.
- 이 표는 매달 한 행이라 두 행이 두 달이다. 불규칙한 거래 두 행은 두 달을 뜻하지 않는다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
change = pd.DataFrame({"sales": monthly})
change["previous"] = monthly.shift(1)
change["change"] = monthly - change["previous"]
change["rolling_2"] = monthly.rolling(2).mean()
print(change)
```

실행 결과

	sales	previous	change	rolling_2
occurred_at				
2026-01-01	55	NaN	NaN	NaN
2026-02-01	45	55.0	-10.0	50.0
2026-03-01	36	45.0	-9.0	40.5

문자열 식으로 행 선택과 열 계산

query: 조건에 맞는 행 선택

- query의 식에서 열 이름은 그대로 쓴다. @threshold는 바깥 Python 변수 15를 참조한다.
- 매출 15 이상이면서 web인 거래는 ID 3·5다.

eval: 열 이름으로 계산식 작성

- eval의 식은 매출/수량을 단가 열에 넣는다. 이 호출은 원본을 바꾸지 않고 새 표를 반환한다.
- 앞 세 단가는 12·9·12.5다. np.allclose로 일반 열 연산과 허용오차 안에서 같은지 검사한다.
- allclose는 값 비교다. 인덱스·열 이름까지 같다는 보장은 아니다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
threshold = 15
selected = sales.query('sales >= @threshold and channel == "web"')
with_price = sales.eval("unit_price = sales / units")
expected = sales["sales"] / sales["units"]
print("선택 ID:", selected["event_id"].tolist())
print("앞 세 단가:", with_price["unit_price"].head(3).tolist())
print(np.allclose(with_price["unit_price"], expected))
```

실행 결과

```
선택 ID: [3, 5]
앞 세 단가: [12.0, 9.0, 12.5]
True
```

큰 중간 배열에서의 eval·query 선택 조건

중간 배열의 계산 비용

- 비교 두 번과 & 결합 한 번으로 bool 배열 3개가 생긴다. 8행이면 배열 데이터는 8+8+8=24 바이트다.
- 1천만 행이면 같은 세 배열은 30,000,000바이트다. nbytes 는 인덱스·문자열 객체·엔진 버퍼를 제외한다.
- 이 값은 배열 데이터량이며 query의 실제 절감량이나 최고 메모리를 뜻하지 않는다.

동치성 확인 후 선택

- NumExpr는 지원하는 수치식을 작은 묶음으로 계산해 큰 임시 배열을 줄이는 엔진이다. 문자열 비교·결과 행 선택 전체를 처리하지는 않는다.
- 8행에서는 일반식도 충분하다. 큰 표에서는 결과가 같은지 확인한 뒤 시간·메모리를 측정해 일반식과 eval·query 중 선택한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
tmp1 = sales["sales"] >= 15
tmp2 = sales["channel"] == "web"
mask = tmp1 & tmp2
print(tmp1.values.nbytes, tmp2.values.nbytes,
      mask.values.nbytes)
```

실행 결과

8 8 8

정리·시간화·요약 순서를 기록하는 통합 파이프라인

개별 기능에서 전달용 요약표 조립으로 전환

- `assign` 은 범주와 시점을 정리하고
`set_index·sort_index` 는 시간 그룹의 기준을 만든다.
- `pd.Grouper(freq="MS")` 는 월, `category` 는 범주로 분할하고 `sum` 은 각 조합의 매출을 합친다.
- `unstack(fill_value=0)` 은 범주를 열로 옮겨 월×범주 3×3 표를 만든다. 앞의 범주×경로 3×2 표와 축이 다르다.

빈 조합과 관측 누락의 구분

- 이 예제는 해당 기간의 거래가 모두 수록됐다고 가정한다. 그래서 1월 snack·3월 drink의 미발생 조합을 0으로 둔다. 기록 누락이라면 0으로 단정할 수 없다.
- `unstack` 은 (월, 범주) 중 범주를 열로 옮긴다. 1월 drink 합계 30은 같은 값을 유지하며 1월 행·drink 열의 셀로 이동한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
prepared = (
    sales.assign(
        category=sales["category"].astype("string").str.strip().str.lower(),
        occurred_at=pd.to_datetime(sales["occurred_at"],
            errors="raise"),
    )
    .set_index("occurred_at")
    .sort_index()
)
plot_data = (
    prepared.groupby([pd.Grouper(freq="MS"), "category"])
    ["sales"]
    .sum().unstack(fill_value=0)
)
print(plot_data)
```

TEXT · 실행 결과

category	drink	food	snack
occurred_at			
2026-01-01	30	25	0
2026-02-01	20	15	10
2026-03-01	0	22	14

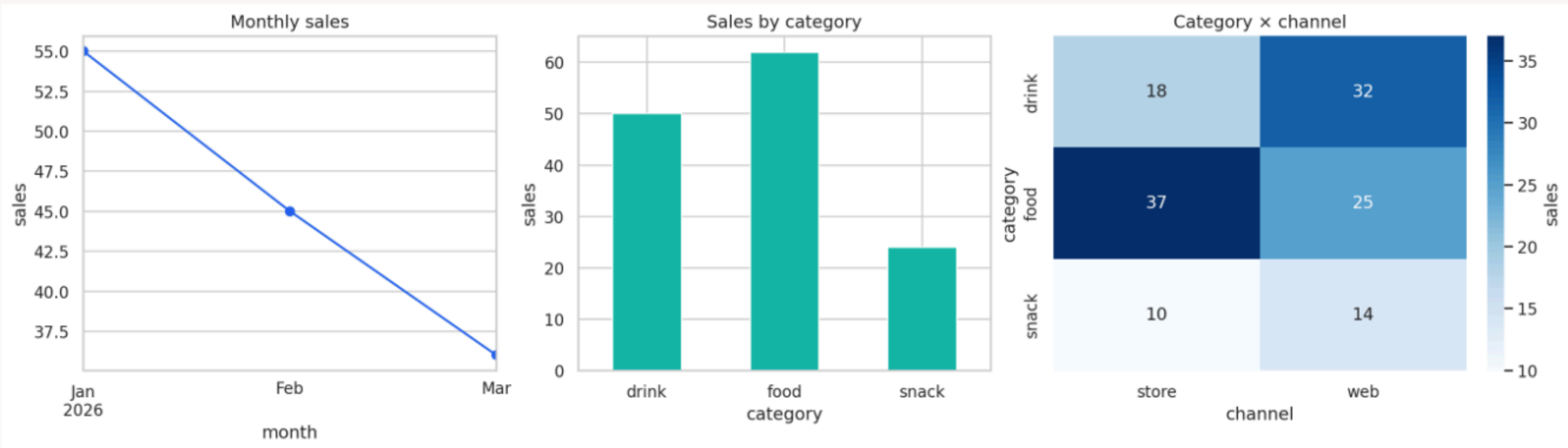
질문에 따른 동일 요약값의 시각화

세 미리보기의 값 연결

- 왼쪽 선은 월별 합계 55 → 45 → 36, 가운데 막대는 범주별 합계 50·62·24 를 나타낸다.
- 오른쪽 색상표는 범주×판매 경로 피벗의 여섯 셀을 색과 숫자로 함께 표시한다.
- 선은 시간 변화, 막대는 범주 크기, 색상표는 두 범주의 조합을 비교하는 질문에 대응한다.

요약표와 미리보기의 입력 관계

- 월별 선은 monthly, 범주별 막대는 totals, 색상표는 matrix 를 받는다. 모두 같은 거래 8행에서 나온 보완 자료다.
- plot_data 는 월×범주 표다. 행 합계 55·45·36, 열 합계 50·62·24 는 앞의 두 요약과 일치하지만 판매 경로는 포함하지 않는다.



검산된 동일 요약값의 선·막대·색상표

작은 불변량을 이용한 전체 결과 검증

검사와 판정 범위

- `assert` 는 조건이 거짓이면 중단한다. `issubset` 은 필수 열 포함, `is_unique` 는 ID 중복 없음을 검사한다.
- `is_monotonic_increasing` 은 시간 오름차순, `hasnans` 는 시간 결측 유무다. `not` 으로 결측 없음을 요구한다.
- `to_numpy().tolist()` 는 표를 행 순서의 값 목록으로 바꾼다. 3x3 모양·합계뿐 아니라 아홉 셀도 기대값과 비교한다.

합계가 놓치는 반례

- 1월 drink 30→31, food 25→24 이면 총합은 여전히 136이다. 합계 보존만으로 누락·중복·분류 오류가 없다고 단정할 수 없다.
- 이 예제의 기대 행렬 비교는 그 오류를 잡는다. 조건이 모두 참이면 성공 문장을 출력하며 실패하면 해당 입력·집계 단계를 조사한다.

분석 노트북 · 순서대로 실행하는 코드 셀

```
required = {"event_id", "occurred_at", "category", "item_tag",
            "channel", "sales", "units"}
assert len(sales) == 8
assert required.issubset(sales.columns)
assert sales["event_id"].is_unique
assert prepared.index.is_monotonic_increasing
assert plot_data.shape == (3, 3)
assert plot_data.to_numpy().sum() == 136
assert sales["sales"].sum() == 136
assert not prepared.index.hasnans
assert plot_data.to_numpy().tolist() == [
    [30, 25, 0], [20, 15, 10], [0, 22, 14]
]
print("8 rows; 3 x 3; total 136; cells OK")
```

실행 결과

8 rows; 3 x 3; total 136; cells OK

데이터 요약과 시계열 처리

Summary

<code>groupby</code>는 키로 분할하고 집계·변환·필터의 서로 다른 출력 계약에 따라 결과를 결합한다.	<code>pivot_table</code>은 두 범주의 집계를 행과 열로 배치하며 주변 합계로 원자료 보존을 검사한다.	벡터화 문자열 연산은 공백·대소문자·구분자를 같은 규칙으로 처리하고 결측 판정을 명시한다.
시간값을 변환·정렬한 뒤 <code>resample</code>·<code>asfreq</code>·<code>shift</code>·<code>rolling</code>의 서로 다른 의미를 구분한다.	<code>query</code>·<code>eval</code>은 식의 결과 동치성과 데이터 크기를 확인한 뒤 선택한다.	구조·키·시간 순서·합계와 셀 값을 검사한 요약표를 선·막대·색상표의 입력으로 사용한다.